

ЛАБОРАТОРНАЯ РАБОТА № 1

Цветовые модели

Цель работы

Изучить основные цветовые модели RGB, CMYK, HSV, HLS, XYZ и CIELAB (Lab), принципы перехода между цветовыми пространствами, понятие цветового охвата (gamut) и проблему отображения цветов, не входящих в диапазон sRGB. Освоить разработку интерактивного приложения, позволяющего выбирать и изменять цвет, одновременно отображая его представления в трёх заданных цветовых моделях: **HSV, XYZ и Lab**.

Задачи работы

В ходе выполнения лабораторной работы были поставлены следующие задачи:

- изучить назначение и область применения цветовых моделей RGB, CMYK, HSV, HLS, XYZ и Lab;
- рассмотреть преобразования между цветовыми пространствами;
- изучить понятие цветового графика МКО, цветового охвата и выхода цвета за границы отображаемого диапазона;
- реализовать приложение с одновременным отображением **HSV, XYZ и Lab**.

Использованные средства разработки

- Python 3.14 — язык реализации математических функций преобразования цветовых моделей;
- Tkinter / ttk — создание интерактивного пользовательского интерфейса;
- стандартная библиотека math, tkinter.colorchooser — математические операции и системный выбор цвета;
- PyCharm — среда разработки проекта;
- unittest — модульное тестирование математических функций.

Ход выполнения работы

1. Разработка интерфейса

Было создано приложение Color Lab на Python с использованием Tkinter. В интерфейсе сформированы панели HSV, XYZ и LAB, а также отдельная карточка предварительного просмотра выбранного цвета.

Для каждого параметра цвета создан переиспользуемый блок, включающий подпись параметра, ползунок `tk.Scale` для плавного изменения и поле `tk.Entry` для точного числового ввода. Таким образом, в каждой из трёх моделей реализованы два способа задания компонентов: ползунок и числовое поле. Третьим способом служит выбор из системной палитры браузера ОС (`colorchooser.askcolor()`), что соответствует требованию о выборе цвета из палитры, аналогично графическим редакторам.

2. Визуальный выбор цвета

Для выбора цвета из палитры добавлена кнопка «Палитра...», вызывающая системный диалог `colorchooser.askcolor()`. Выбранный цвет поступает в виде кортежа RGB, после чего выполняется перевод $RGB \rightarrow HSV$, и далее запускается общий механизм пересчёта цветовых моделей.

3. Архитектура приложения

Код разделён на независимые слои:

- **Model** — чистая математика цветовых преобразований, не знающая об интерфейсе. Представлена модулями `color_math.py`, `standards.py`, `converter.py`.
- **View** — только визуальное представление, кнопки, ползунки, поля ввода. Представлен модулем `main_window.py`.
- **ViewModel** — связующее звено, которое принимает данные от интерфейса, передаёт их в математику и возвращает результат обратно. Представлен модулем `color_viewmodel.py`.

4. Реализация структур и цветовых моделей

В качестве эталонного состояния в `ViewModel` хранится **HSV** в виде трёх значений типа `float`. Модели **XYZ** и **LAB** не хранятся постоянно, а каждый раз вычисляются из **HSV** при запросе. Это исключает накопление ошибок округления и рассинхронизацию моделей.

В модели определены следующие преобразования:

- $HSV \rightarrow RGB$;
- $RGB \rightarrow HSV$;
- $RGB \rightarrow XYZ$;
- $XYZ \rightarrow RGB$ (с обработкой выхода за пределы диапазона);
- $XYZ \rightarrow LAB$;
- $LAB \rightarrow XYZ$.

Преобразования $HSV \leftrightarrow LAB$ выполняются через **RGB** и **XYZ** как промежуточные пространства, поскольку прямые формулы для этой пары отсутствуют.

5. Проверка диапазонов и округление

В приложении добавлена функция нормализации значений, которая приводит введённые числа к допустимым диапазонам:

- **HSV:** H — по модулю 360 (при вводе 500 получается 140, при вводе -30 получается 330), S и V — в диапазоне 0...1;
- **XYZ:** X — 0...95.05, Y — 0...100, Z — 0...108.9;
- **LAB:** L — 0...100, a и b — от -128 до 127.

Если пользователь вводит некорректное значение (например, abc), поле возвращается к предыдущему корректному значению. Также реализовано округление значений: HSV отображается с точностью до двух знаков, XYZ и LAB — с точностью до двух знаков после запятой.

6. Автоматические тесты

Для проверки математики написаны модульные тесты (модуль tests/test_color_math.py):

- проверка перевода чистого красного RGB(255, 0, 0) в LAB — ожидаемые координаты (53.24, 80.09, 67.20) при D65;
- проверка перевода белого RGB(255, 255, 255) в XYZ — должна получиться точка белого D65 (95.05, 100.0, 108.9);
- проверка кругового перехода HSV → RGB → HSV;
- проверка срабатывания флага clipping при выходе XYZ за диапазон sRGB;
- проверка стратегии scaling.

Все тесты запускаются командой `python -m unittest discover tests` и успешно проходят.

Проблемы, возникшие в работе, и способы решения

1. Рекурсивный вызов обработчика ползунка

Причина: `ttk.Scale.set()` при программном обновлении значения тоже вызывает свой `command`, что приводило к бесконечной рекурсии при пересчёте.

Решение: введён флаг `_updating`, который устанавливается в `True` на время `_refresh()` и в `False` по завершении. Обработчик движения ползунка игнорирует вызовы, если флаг установлен.

2. Ползунок выходил за пределы блока

Причина: значение из математики (например, $X = 95.0500001$) записывалось в `scale.set()` как есть, и Tkinter рисовал ползунок за границей.

Решение: перед `scale.set()` значение нормализуется в диапазон `[lo, hi]` конкретного слайдера. Границы `lo` и `hi` хранятся рядом со слайдером в словаре.

3. Некорректный ввод в числовое поле

Причина: при вводе значения вне диапазона поле не обновлялось визуально до следующего пересчёта, а при вводе мусора — вылетала ошибка парсинга.

Решение: добавлена функция нормализации `_commit_entry`, которая после парсинга приводит значение к допустимому диапазону (для `H` — по модулю 360, для остальных — `clamp`), применяет его и полностью обновляет все блоки. При вводе мусора просто вызывается `_refresh()` и поле возвращается к предыдущему корректному значению.

Результаты работы

В результате выполнения лабораторной работы создано приложение **Color Lab**, которое:

- отображает выбранный цвет в **HSV, XYZ и LAB**;
- позволяет вводить точные значения компонентов через числовые поля;
- позволяет плавно менять компоненты с помощью ползунков;
- позволяет выбирать цвет из системной палитры;
- автоматически пересчитывает другие модели при изменении любого параметра;
- реализует стратегию **Clipping** при выходе цвета за пределы sRGB;

Вывод

Практическим результатом стало создание интерактивного приложения, в котором реализованы все основные способы задания цвета: ползунки, числовые поля и выбор из палитры. Для перевода между моделями использовался промежуточный формат XYZ и линеаризация RGB-пространства, так как напрямую перевести, например, HSV в LAB невозможно — все преобразования проходят через RGB и XYZ. Матрицы перехода $RGB \leftrightarrow XYZ$ не прописаны жёстко, а рассчитываются в коде, включая ручное вычисление обратной матрицы.